



Programming for Artificial Intelligence Lab

Submitted to:

Rasikh Ali

Submitted by:

Shazra Zainab 003

Department:

Software Engineering

Section:

BSAI(4A)

Roll No:

SU92-BSAIM-F23-003

Task 09

Text Representation or Text Embedding Techniques in NLP

Implementing One-Hot Encoding Using Python:

Input:

```
one_hot_encoding.py > ...
1  import pandas as pd
2  from sklearn.preprocessing import OneHotEncoder
3  # Original data
4  data = {
5      'Employee id': [10, 20, 15, 25, 30],
6      'Gender': ['M', 'F', 'F', 'M', 'F'],
7      'Remarks': ['Good', 'Nice', 'Good', 'Great', 'Nice']
8  }
9
10 df = pd.DataFrame(data)
11 print(f"Original Employee Data:\n{df}\n")
12
13 df_pandas_encoded = pd.get_dummies(df, columns=['Gender', 'Remarks'], drop_first=True)
14 print(f"One-Hot Encoded Data using Pandas:\n{df_pandas_encoded}\n")
15
16 categorical_columns = ['Gender', 'Remarks']
17 encoder = OneHotEncoder(sparse_output=False)
18 one_hot_encoded = encoder.fit_transform(df[categorical_columns])
19
20 one_hot_df = pd.DataFrame(one_hot_encoded,
21                           columns=encoder.get_feature_names_out(categorical_columns))
22
23 df_sklearn_encoded = pd.concat([df.drop(categorical_columns, axis=1), one_hot_df], axis=1)
24 print(f"One-Hot Encoded Data using Scikit-Learn:\n{df_sklearn_encoded}\n")
25
```

Output:

```
PS C:\Users\DELL\Desktop\PAI Lab Task 09> & "C:/Users/DELL/Desktop/PAI Lab Task 09/.venv/Scripts/python.exe"
Task 09/one_hot_encoding.py"
Original Employee Data:
  Employee id Gender Remarks
0          10      M    Good
1          20      F    Nice
2          15      F    Good
3          25      M   Great
4          30      F    Nice

One-Hot Encoded Data using Pandas:
  Employee id Gender_M Remarks_Great Remarks_Nice
0          10      True         False         False
1          20     False         False          True
2          15     False         False         False
3          25      True          True         False
4          30     False         False          True

One-Hot Encoded Data using Scikit-Learn:
  Employee id Gender_F Gender_M Remarks_Good Remarks_Great Remarks_Nice
0          10      0.0      1.0          1.0          0.0          0.0
1          20      1.0      0.0          0.0          0.0          1.0
2          15      1.0      0.0          1.0          0.0          0.0
3          25      0.0      1.0          0.0          1.0          0.0
4          30      1.0      0.0          0.0          0.0          1.0

PS C:\Users\DELL\Desktop\PAI Lab Task 09>
```

Bag of words (BoW) model in NLP

Input:

```
one_hot_encoding.py  bow.py X
bow.py > ...
1  import nltk
2  import re
3  import numpy as np
4  # Download necessary tokenizer
5  nltk.download('punkt')
6  # Sample text
7  text = """My name is Shazra Zainab"""
8  # Tokenize into sentences
9  dataset = nltk.sent_tokenize(text)
10 # Preprocess each sentence
11 for i in range(len(dataset)):
12     dataset[i] = dataset[i].lower()           # convert to lowercase
13     dataset[i] = re.sub(r'\W', ' ', dataset[i]) # remove non-word characters
14     dataset[i] = re.sub(r'\s+', ' ', dataset[i]) # remove extra spaces
15 # Print the final cleaned text
16 print("Preprocessed Sentences:")
17 print(dataset)
18
```

Output:

```
PS C:\Users\DELL\Desktop\PAI Lab Task 09> & "C:/Users/DELL/Desktop/PAI Lab Task 09/.venv/Scripts/python.exe" "C:/Users/DELL/Desktop/PAI Lab Task 09/bow.py"
[nltk_data] Downloading package punkt to
[nltk_data]   C:\Users\DELL\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt is already up-to-date!
Preprocessed Sentences:
['my name is shazra zainab']
PS C:\Users\DELL\Desktop\PAI Lab Task 09>
```

N-Gram Language Modelling with NLTK

Input:

```
ngram.py > ...
1  # Import necessary libraries
2  import nltk
3  from nltk import bigrams, trigrams
4  from nltk.corpus import reuters
5  from collections import defaultdict
6  # Download necessary NLTK resources
7  nltk.download('reuters')
8  nltk.download('punkt')
9  # Tokenize the text
10 words = nltk.word_tokenize(' '.join(reuters.words()))
11 # Create trigrams
12 tri_grams = list(trigrams(words))
13 # Build a trigram model
14 model = defaultdict(lambda: defaultdict(lambda: 0))
15 # Count frequency of co-occurrence
16 for w1, w2, w3 in tri_grams:
17     model[(w1, w2)][w3] += 1
18 # Transform the counts into probabilities
19 for w1_w2 in model:
20     total_count = float(sum(model[w1_w2].values()))
21     for w3 in model[w1_w2]:
22         model[w1_w2][w3] /= total_count
```

```

2         model[w1_w2][w3] /= total_count
3     # Function to predict the next word
4     def predict_next_word(w1, w2):
5         """
6         Predicts the next word based on the previous two words using the trained trigram model.
7         Args:
8         w1 (str): The first word.
9         w2 (str): The second word.
10
11         Returns:
12         str: The predicted next word.
13         """
14         next_word = model[w1, w2]
15         if next_word:
16             predicted_word = max(next_word, key=next_word.get) # Choose the most likely next word
17             return predicted_word
18         else:
19             return "No prediction available"
20
21     # Example usage
22     print("Next Word:", predict_next_word('the', 'stock'))

```

Output:

```

● PS C:\Users\DELL\Desktop\PAI Lab Task 09> & "C:/Users/DELL/Desktop
Task 09/n_gram.py"
[nltk_data] Downloading package reuters to
[nltk_data]      C:\Users\DELL\AppData\Roaming\nltk_data...
[nltk_data] Package reuters is already up-to-date!
[nltk_data] Downloading package punkt to
[nltk_data]      C:\Users\DELL\AppData\Roaming\nltk_data...
[nltk_data] Package punkt is already up-to-date!
Next Word: of
● PS C:\Users\DELL\Desktop\PAI Lab Task 09> & "C:/Users/DELL/Desktop

```

Understanding TF-IDF (Term Frequency-Inverse Document Frequency)

Input:

```

tf_idf.py > ...
1  from sklearn.feature_extraction.text import TfidfVectorizer
2
3  # Sample documents
4  documents = [
5      "My name is Shazra Zainab",
6      "I love learning machine learning",
7      "Python is great for data science"
8  ]
9
10 # Create and fit the vectorizer
11 tfidf = TfidfVectorizer()
12 tfidf_matrix = tfidf.fit_transform(documents)
13
14 # Now this will work:
15 print("Word indexes:")
16 print(tfidf.vocabulary_)
17

```

Output:

```

PS C:\Users\DELL\Desktop\PAI Lab Task 09> & "C:/Users/DELL/Desktop/PAI Lab Task 09/.env/Scripts/python.exe" "c:/Users/DELL/Desktop/PAI Lab
Task 09/tf_idf.py"
Word indexes:
{'my': 7, 'name': 8, 'is': 3, 'shazra': 11, 'zainab': 12, 'love': 5, 'learning': 4, 'machine': 6, 'python': 9, 'great': 2, 'for': 1, 'data':
0, 'science': 10}
PS C:\Users\DELL\Desktop\PAI Lab Task 09>

```

N-Gram Language Modelling with NLTK

Input:

 n_gram.py > ...

```
1  # Import necessary Libraries
2  import nltk
3  from nltk import bigrams, trigrams
4  from nltk.corpus import reuters
5  from collections import defaultdict
6  # Download necessary NLTK resources
7  nltk.download('reuters')
8  nltk.download('punkt')
9  # Tokenize the text
10 words = nltk.word_tokenize(' '.join(reuters.words()))
11 # Create trigrams
12 tri_grams = list(trigrams(words))
13 # Build a trigram model
14 model = defaultdict(lambda: defaultdict(lambda: 0))
15 # Count frequency of co-occurrence
16 for w1, w2, w3 in tri_grams:
17     model[(w1, w2)][w3] += 1
18 # Transform the counts into probabilities
19 for w1_w2 in model:
20     total_count = float(sum(model[w1_w2].values()))
21     for w3 in model[w1_w2]:
22         model[w1_w2][w3] /= total_count
```

```

2         model[w1_w2][w3] /= total_count
3     # Function to predict the next word
4     def predict_next_word(w1, w2):
5         """
6         Predicts the next word based on the previous two words using the trained trigram model.
7         Args:
8         w1 (str): The first word.
9         w2 (str): The second word.
10
11         Returns:
12         str: The predicted next word.
13         """
14         next_word = model[w1, w2]
15         if next_word:
16             predicted_word = max(next_word, key=next_word.get) # Choose the most likely next word
17             return predicted_word
18         else:
19             return "No prediction available"
20
21     # Example usage
22     print("Next Word:", predict_next_word('the', 'stock'))

```

Output:

```

0, 'science': 10}
● PS C:\Users\DELL\Desktop\PAI Lab Task 09> & "C:/Users/DELL/Desktop/PAI
Task 09/N_gram2.py"
[nltk_data] Downloading package reuters to
[nltk_data] C:\Users\DELL\AppData\Roaming\nltk_data...
[nltk_data] Package reuters is already up-to-date!
[nltk_data] Downloading package punkt to
[nltk_data] C:\Users\DELL\AppData\Roaming\nltk_data...
[nltk_data] Package punkt is already up-to-date!
Next Word: of
○ PS C:\Users\DELL\Desktop\PAI Lab Task 09> 

```

